

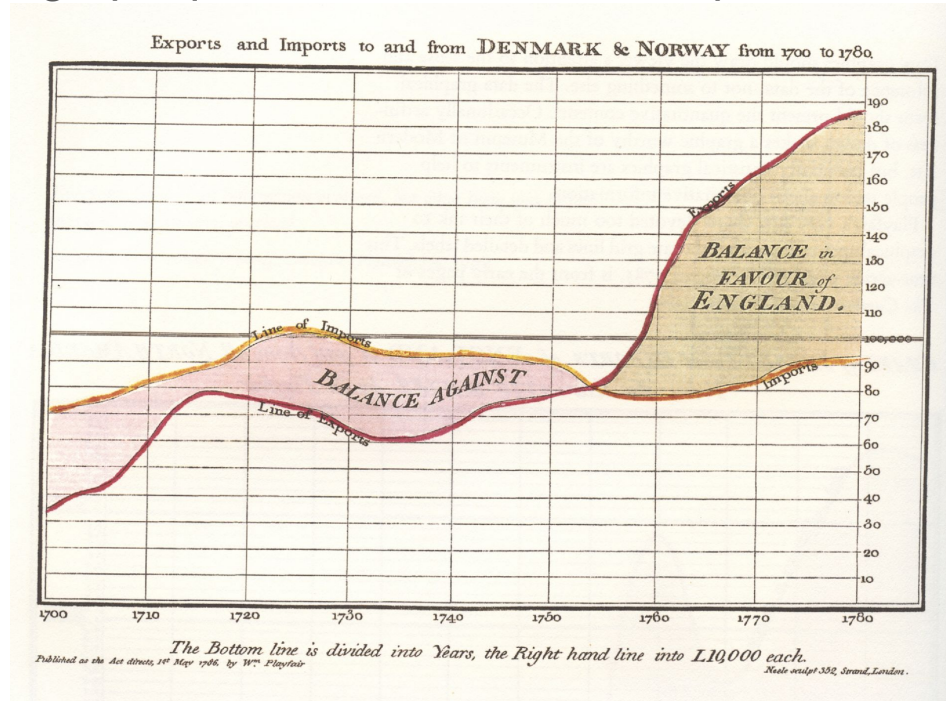
Concevoir une dataviz

Rappels et généralités

Jean Delpech
Data Visualization - Ynov M2 IA/Data
Octobre 2026

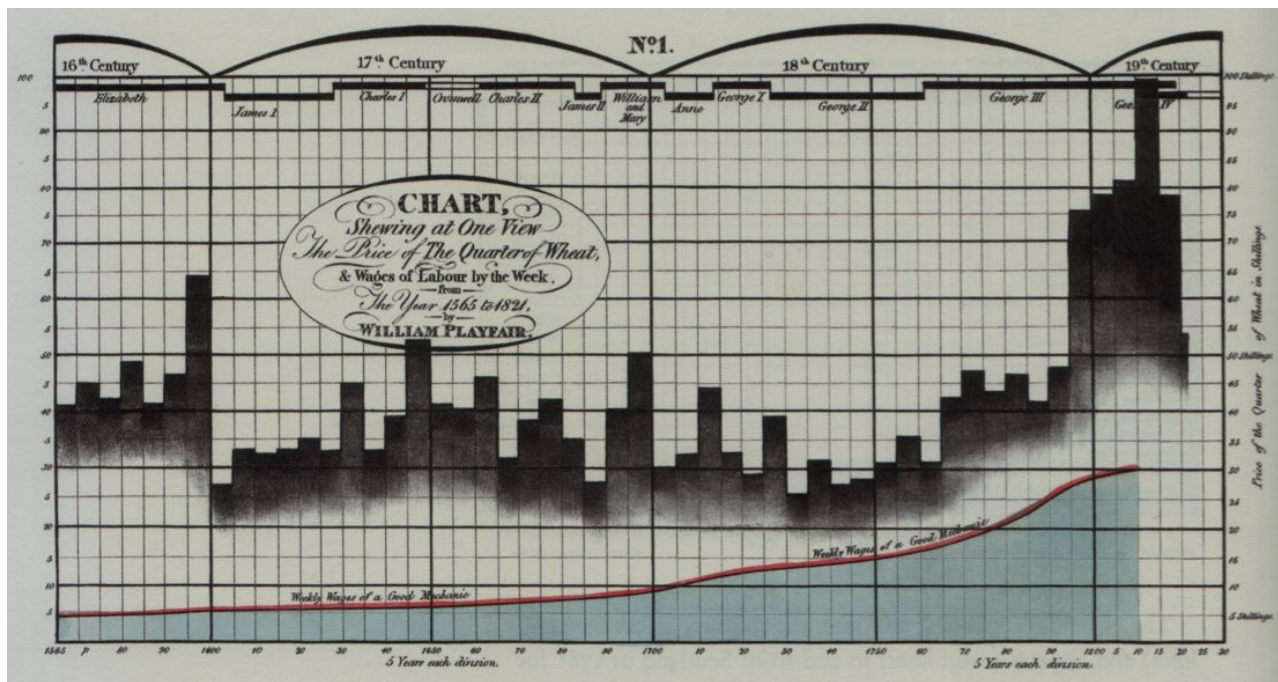
Historique (1) : représenter les données

- 1786 [William Playfair](#) : graphique en barre, courbe temporelle



Historique (1) : représenter les données

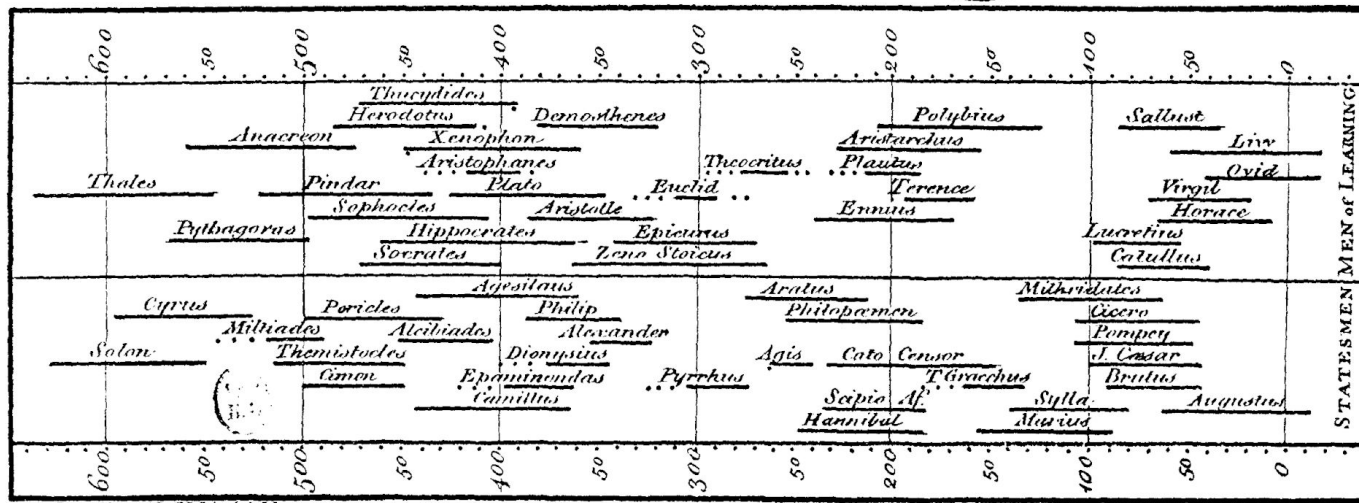
- 1786 [William Playfair](#) : graphique en barre, courbe temporelle



Historique (1) : représenter les données

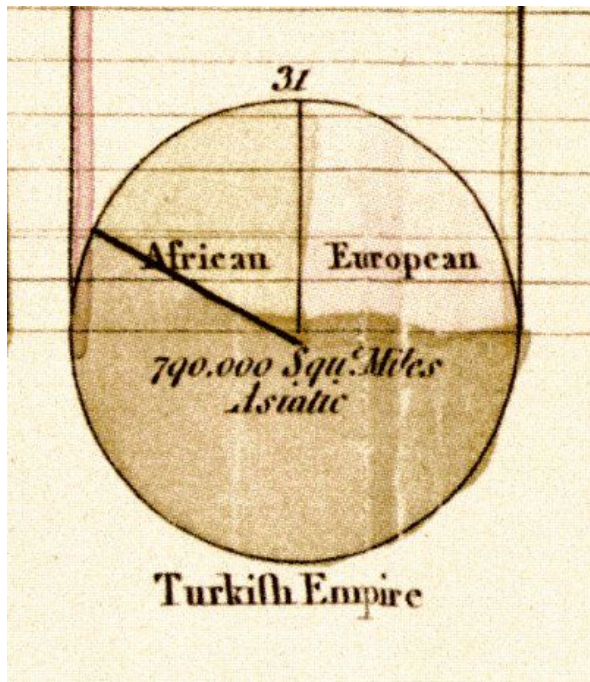
- 1786 [William Playfair](#) : graphique en barre, courbe temporelle

A Specimen of a Chart of Biography.



Historique (1) : représenter les données

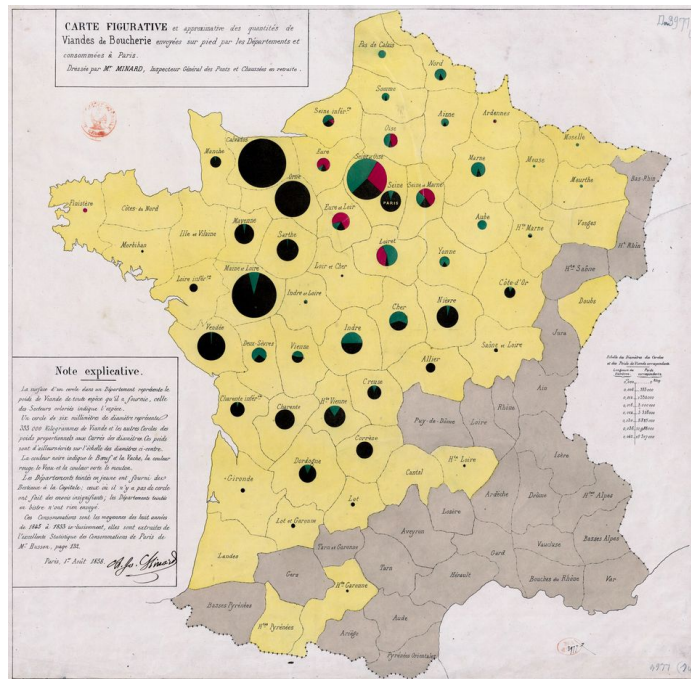
- 1786 [William Playfair](#) : pie-chart/camembert



[Playfair Prize](#) : prix pour les étudiants en datajournalisme

Historique (1) : représenter les données

- 1858 [Charles Joseph Minard](#) : provenance viande consommée à Paris

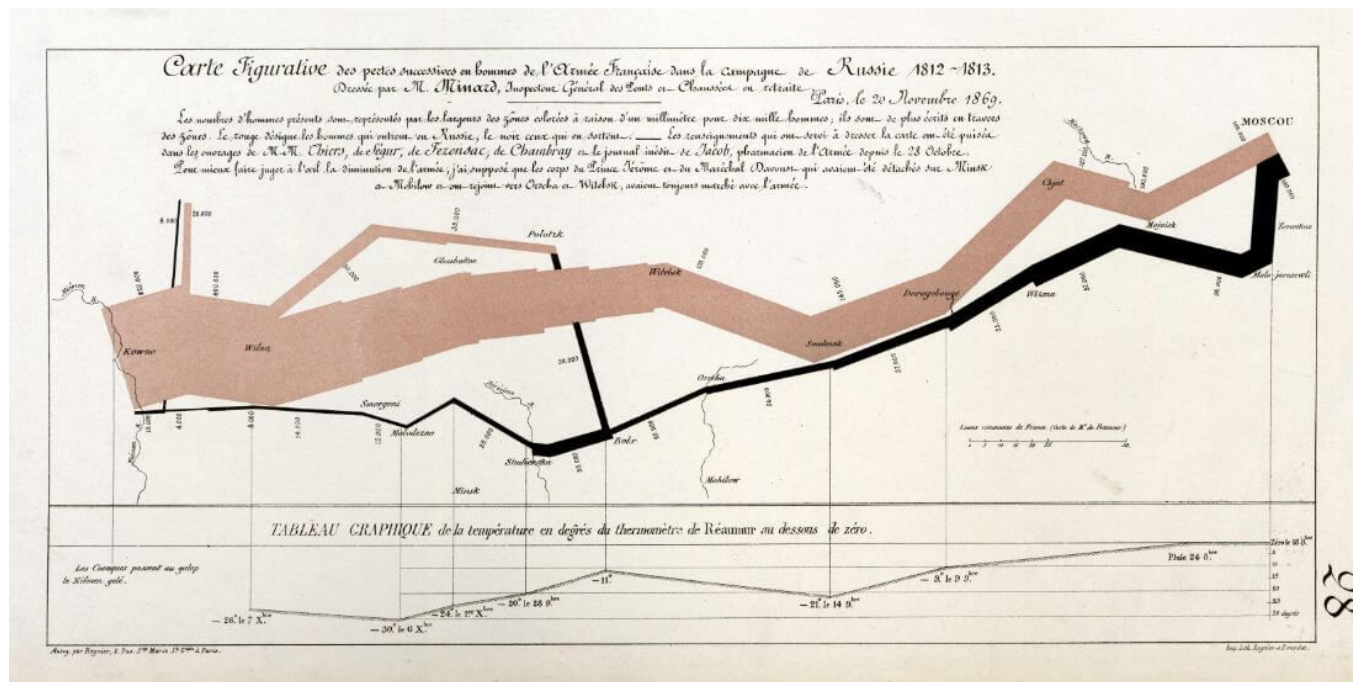


Historique (1) : représenter les données

- 1869 Charles Joseph Minard : cartographie, diagramme de Sankey

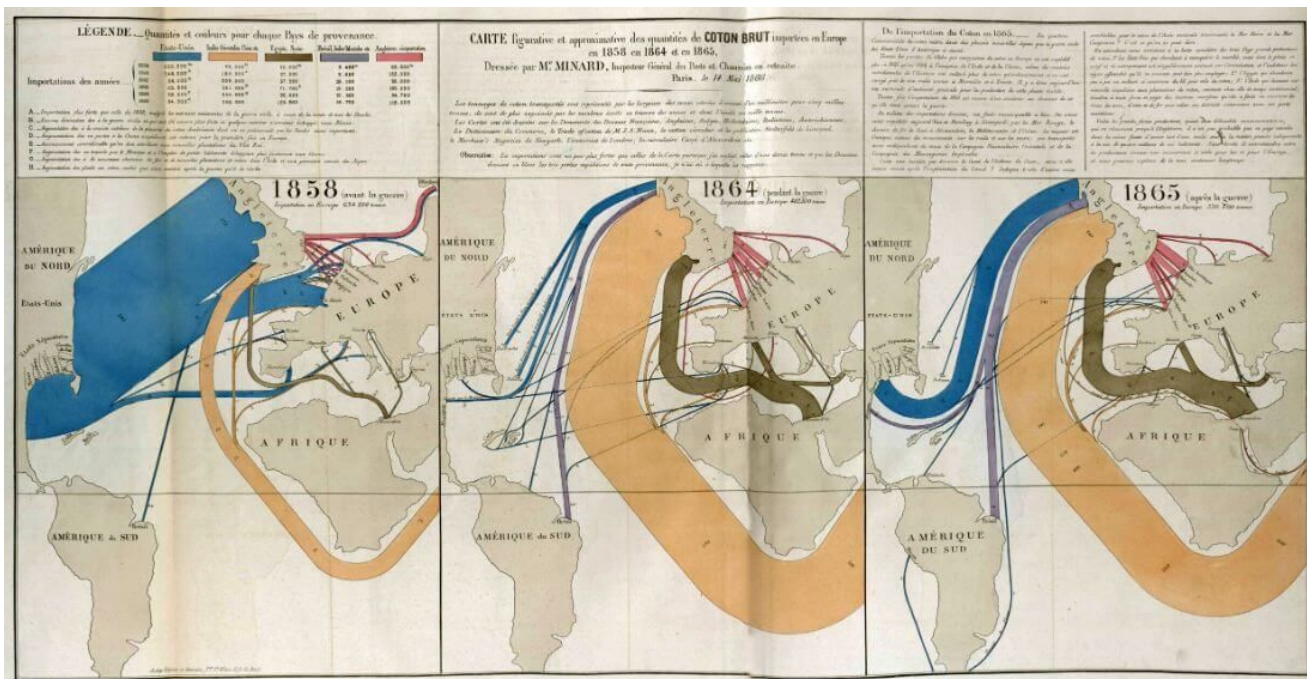
6 variables en 2D :

- nombre de troupes
- température
- distance
- lat/long (carte)
- direction
- dates



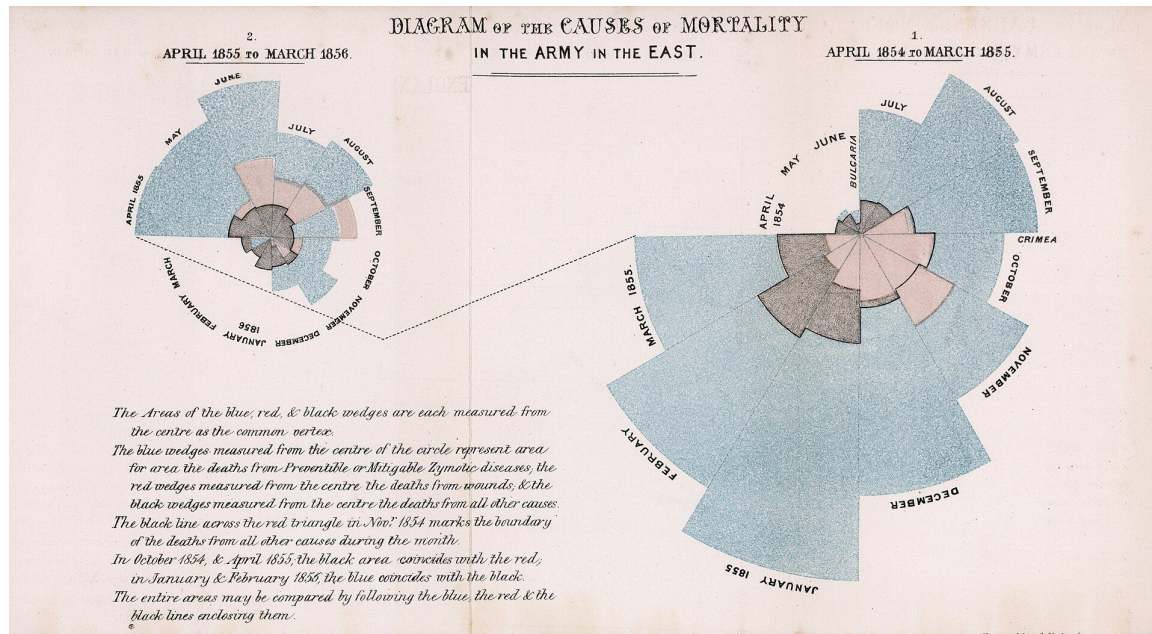
Historique (1) : représenter les données

- 1865 [Charles Joseph Minard](#) : import/export coton



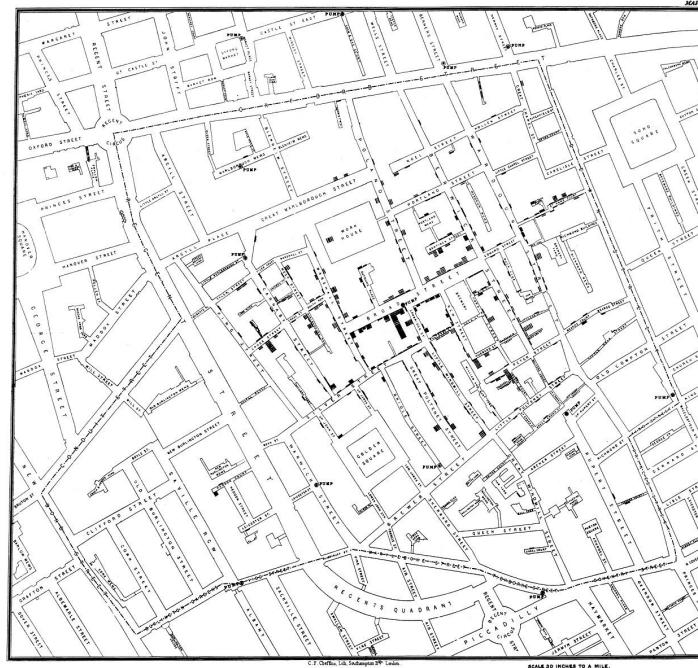
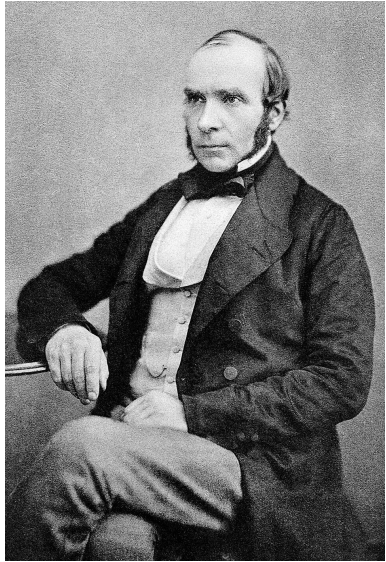
Historique (2) : dataviz pour prendre une décision

- 1856 [Florence Nightingale](#) : mortalité hospitalière dans les armées (Crimée), histogrammes circulaires



Historique (2) : dataviz pour prendre une décision

- 1854 [John Snow](#) : épidémie de choléra Londres, cartographie



Historique (3) : formalisation



- [John Wilder Tukey](#) : [*Exploratory Data Analysis*](#), 1977

- [Edward Tufte](#) : [*The Visual Display of Quantitative Information*](#), 1983



Historique (4) : dataviz / infographie – web interactif

- web interactif : javascript, [D3.js](#) (2011)
- Tableau...
- grammaire de visualisation Vega-lite
- framework Python : Streamlit (), Dash (), etc.

Historique (5) : références

- Michael Friendly, [*A Brief History of Data Visualization*, 2007](#)
- Edward Tufte : [*The Visual Display of Quantitative information*, 1983](#)

Rappel (1) : types de graphique

Il est important de créer des dataviz congruente avec le type de données.

Attention, ce ne sont pas des règles absolues : l'objectif est de faciliter la compréhension en mettant en cohérence le **format** de la donnée et l'**analyse** effectuée.

Le contexte et le **message** restent essentiels.

Par exemple à partir de données temporelles :

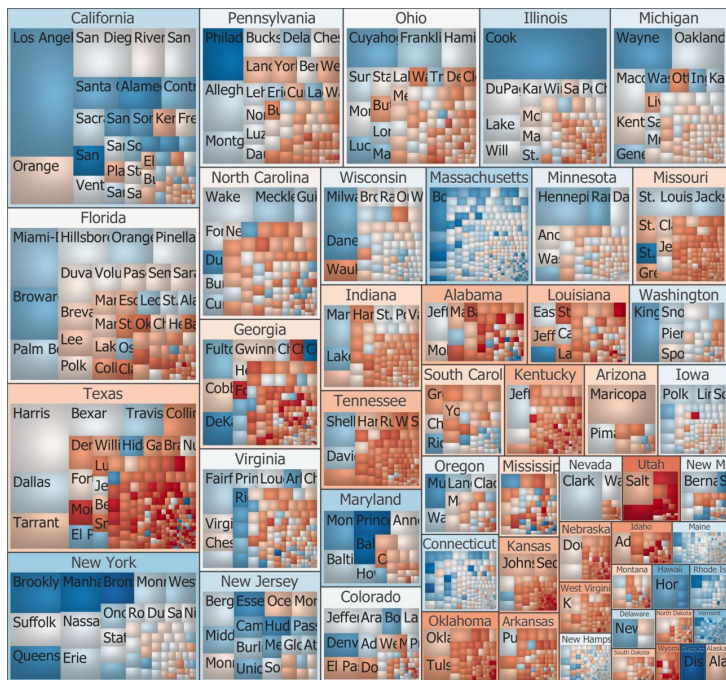
- en général séries temporelles = courbes pour traduire une tendance continue
- mais on peut faire des graphiques en barres pour comparer des périodes (saisons ≈ catégories)

Rappel (1) : types de graphique

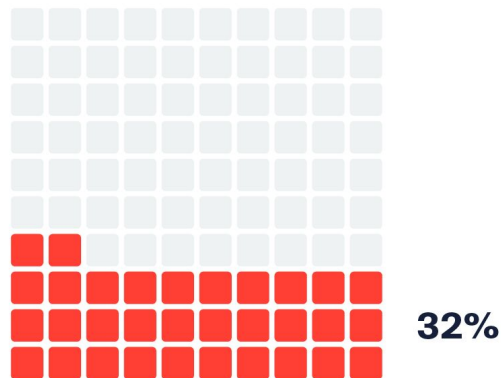
Type de donnée	Type de graphe
Quantitatif continu (1 variable)	Histogramme, densité, boîte à moustaches
Quantitatif (2 variables)	Nuage de points, heatmap de corrélation
Catégoriel	Barres, treemap , waffle chart
Temporel	Courbe, graphique en aires, chronologie
Relationnel / réseau	Graphe de nœuds-liens, diagramme de Sankey
Hiérarchique	Treemap, dendrogramme , sunburst
Géospatial	Carte choroplèthe, carte de points, carte de flux

Rappel (1) : types de graphique

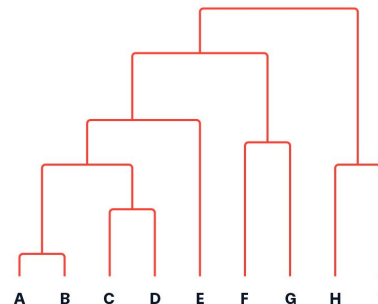
Treemap



Waffle chart (« gauffre »)



Dendrogramme (*δένδρον* *déndron* « arbre »)



Rappel (1) : types de graphique (ressources)

- <https://www.data-to-viz.com/> classe les graphiques par format de données
- [« Vocabulaire visuel » du Financial Time](#) (dépôt github), une ressource pour aider les datajournalistes à choisir la bonne sémiologie. La classification ici n'est pas par format, mais par les patterns que l'on souhaite mettre en valeur (comparaison, distribution, évolution, part de l'ensemble...)
- <https://datavizproject.com/> un simple catalogue de graphiques (pas d'aide à la décision), par types

Rappel (2) : data storytelling

- On ne peut se contenter de juste montrer des données
- On cherche à structurer une **analyse** autour d'un **message**

contexte → problème → preuve visuelle → action ou solution (conclusion)

- Il faut mâcher le travail de réflexion du lecteur / de l'utilisateur
- Ce n'est pas au lecteur de reconstituer le sens de vos graphiques pour comprendre / retrouver vos conclusions

Référence : Cole Nussbaumer Knaflitz, *Storytelling with Data: A Data Visualization Guide for Business Professionals*, Wiley, 2015

<https://www.storytellingwithdata.com/>

Perception, cognition, ergonomie

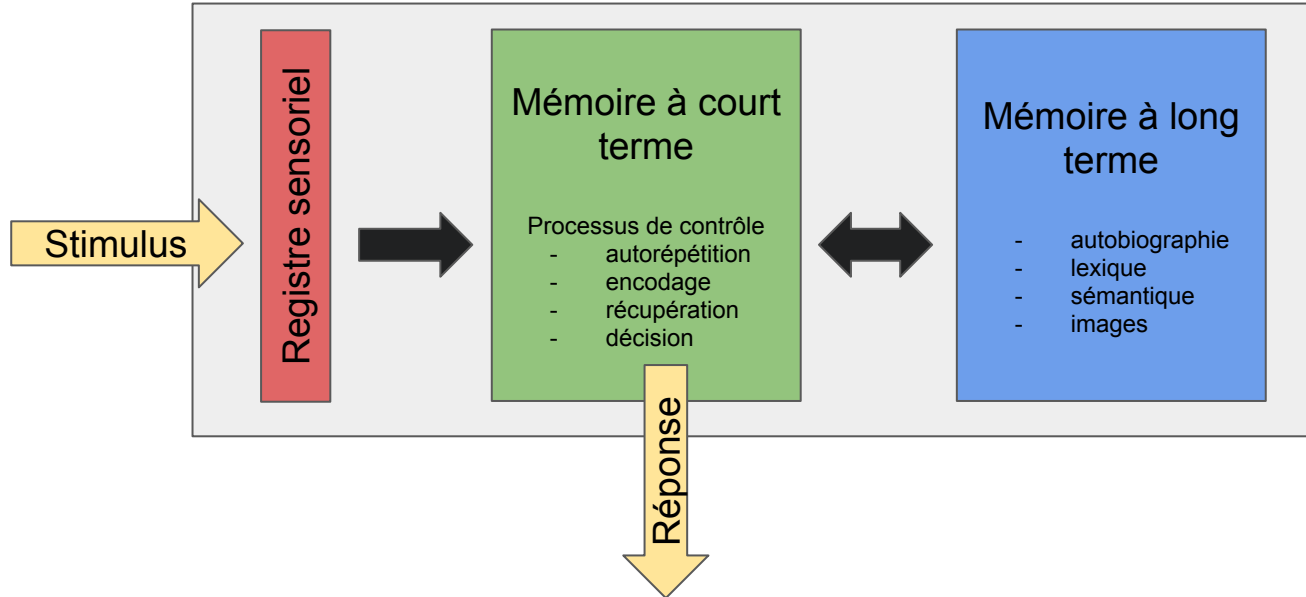
- Pourquoi certains choix de design fonctionnent mieux que d'autres ?
- La réponse se trouve dans le fonctionnement du système visuel et la cognition humaines

Perception (1) : processus préattentifs

- processus automatiques
- durée <200/250ms
 - couleur / teinte
 - orientation
 - taille
 - forme (grossière)
 - mouvement / direction - vitesse
 - position
- encodage qui repose sur des attributs « préattentifs » (i.e. ronds rouges vs triangles bleus) est remarqué instantanément
- encodage qui demande d'analyser formes complexes demande de faire intervenir des processus attentionnels contrôlés, donc du temps et de l'effort

Perception (2) : empan mnésique

Atkinson et Schiffrin (1968) : obsolète aujourd'hui mais donne un cadre



voir les articles suivants sur Wikipedia : [le présent modèle](#), [la mémoire à court terme](#), [la mémoire de travail](#), [la mémoire à long terme](#), [la mémoire sensorielle](#)

Perception (2) : empan mnésique

- [Loi de Miller](#) (1956) ou
- « Le nombre magique 7, plus ou moins 2 » (voire moins cf. Farrington 2011)
- L'idée principale est que notre [capacité mnésique à court terme et attentionnelle](#), est très limitée
- une dataviz/un dashboard qui affiche 20 indicateurs d'un coup dépasse ce que l'utilisateur peut réellement retenir et comparer.

Références

- [Colin Ware, *Information Visualization: Perception for Design*, 2015](#)
- [Healey & Enns, *Attention and Visual Memory in Visualization and Computer Graphics*, 2012](#)

Perception (3) : hiérarchisation des encodages

- [Cleveland & McGill \(1984\)](#) : on peut classer les encodages visuels par précision de décodage par l'œil humain

**Position sur une échelle commune > Position sur échelles non alignées >
Longueur > Angle/Pente > Aire > Volume/Densité > Couleur (teinte)**

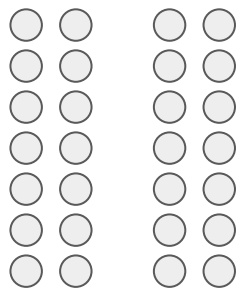
- Exemple : un nuage de points (position) permet une comparaison plus précise qu'un camembert (angle), qui lui-même est plus précis qu'une comparaison de tailles de bulles (aire).

Perception (4) : principes de la Gestalt

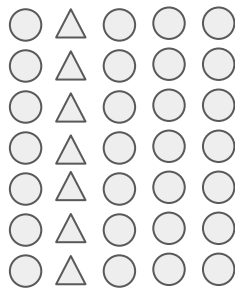
La psychologie de la Gestalt (école allemande du début du 20e siècle) décrit comment nous regroupons spontanément des éléments visuels en unités perçues comme cohérentes, avant même toute analyse consciente.

Gestalt : lois

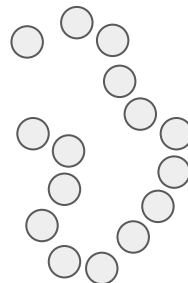
Proximité



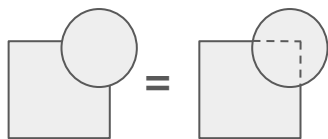
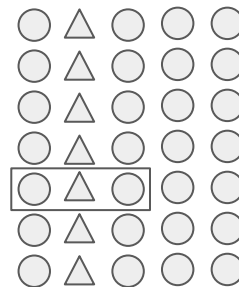
Similarité



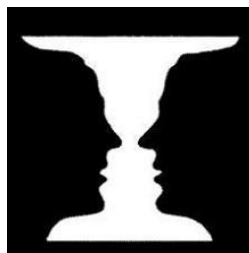
Continuité



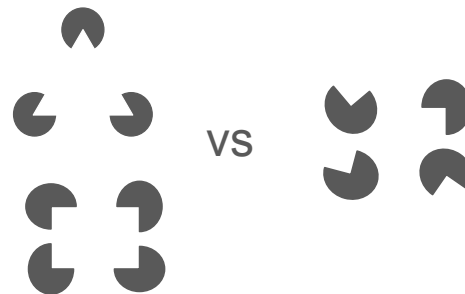
Région commune



Prägnanz



Forme/fond



Fermeture

Perception (4) : pincipes de la Gestalt

- **Proximité** : des éléments rapprochés sont perçus comme liés
→ les layouts de dashboard sont organisés en blocs logiques
- **Similarité** : des éléments de même couleur/forme sont perçus comme un même groupe
→ choisir une palette catégorielle cohérente
- **Clôture** : on complète mentalement des formes incomplètes
→ permet d'alléger un graphique sans fermer chaque contour
- **Continuité** : l'œil suit les lignes et alignements existants
→ on aligne les axes dans un graphe à facettes (*small-multiple*)

Au cœur de la conception : tâches analytiques

Il faut se demander **quelle tâche** on souhaite permettre au lecteur d'accomplir :

- identifier une valeur ?
- comparer des catégories ?
- repérer une corrélation ?
- détecter une tendance ?
- filtrer un sous-ensemble ?...

À lire (liste de 10 tâches analytiques de bas niveau) :

[Robert Amar, James Eagan & John Stasko, Low-Level Components of Analytic Activity in information Visualization, IEEE InfoVis, 2005](#)

Pour aller plus loin : [cours de Tamara Munzner](#) (UBC, liens vers sa chaîne YT)

Application au dashboarding

Nous verrons en détail le jour 3 l'application au dashboarding. C'est assez simple et de « bon sens » maintenant que vous avez ce fondement théorique :

- **limiter la densité informationnelle** (empan mnésique)
- **regroupement logique** des indicateurs liés (proximité/similarité)
- usage des attributs préattentifs pour **guider l'œil** vers ce qui compte (une alerte en rouge saturé plutôt qu'un simple changement de nuance)
- **réduction du bruit** visuel (tout élément qui ne sert ni la lecture ni la tâche a un coût cognitif, fût-il esthétique).

Un peu de technique : grammaires de visualisation

Ici il s'agira surtout d'une présentation théorique, conceptuelle, sans vraiment de code, pour une prise en main pratique [un notebook](#) complète cette présentation.

Note : plus généralement plusieurs notebooks figurent dans le dépôt, issus de mes autres cours, pour prendre en main certaines technos ou concepts (géodata, Dash/Plotly, Superset, Metabase...). N'hésitez pas à les explorer aussi selon vos besoins.

Grammaires : principe général

Une **grammaire de visualisation** part du constat que tout graphique statistique peut se décomposer en une poignée de composants indépendants et combinables :

- les données
- un mapping (ou encodage) : quelle colonne de données va sur quel canal visuel (position, couleur, taille, forme...)
- une transformation statistique / agrégation éventuelle (moyenne, régression, densité...)
- une forme géométrique qui représente chaque observation (point, barre, ligne...)

Grammaires : principe général

Une **grammaire de visualisation** part du constat que tout graphique statistique peut se décomposer en une poignée de composants indépendants et combinables :

- une échelle qui traduit une valeur de donnée en valeur visuelle
- un système de coordonnées
- un « facettage » (répétition en *small-multiples*)
- une charte graphique (thème), indépendante des données elles-mêmes

Grammaires : principe général

- Conséquence directe : au lieu de se demander :
« *quel type de graphique dois-je choisir ?* », on se demande :
« *quelles variables encoder, et sur quels canaux ?* »
- Un nuage de points coloré et un histogramme empilé ne sont alors plus deux objets différents, mais deux combinaisons différentes des mêmes briques.
- Livre fondateur :
Leland Wilkinson, *The Grammar of Graphics*, 2005
- Article qui a développé et popularisé l'idée :
Hadley Wickham, [*A Layered Grammar of Graphics*](#), 2010

Grammaires : ggplot2

- La première implémentation largement adoptée de cette grammaire est *ggplot2*, en **R** (Hadley Wickham, à partir de 2007)
- Elle est aujourd'hui omniprésente en recherche, en statistique appliquée, et en datajournalisme
- la BBC par exemple, a construit [bbplot](#) un habillage de *ggplot2* utilisé en production par son équipe Visual and Data Journalism depuis 2018.

Site officiel : ggplot2.tidyverse.org

Grammaires : Altair/Vega-Lite vs Plotnine (Python)

Deux bibliothèques Python implémentent cette même idée, avec des philosophies différentes :

- **Plotnine** est un portage direct de *ggplot2* en Python
- **Altair** génère des spécifications **Vega-Lite**, une grammaire JSON déclarative développée à l'université de Washington, qui reprend les mêmes briques mais y ajoute une dimension absente de *ggplot2/Plotnine* : une **grammaire de l'interaction**
- Altair est aussi un pont vers l'écosystème JavaScript. C'est une spécification JSON exécutée par un moteur de rendu en JavaScript (Vega)

Grammaires : Altair/Vega-Lite vs Plotnine (réf.)

- Article fondateur de Vega-Lite : [Satyanarayan, Moritz, Wongsuphasawat & Heer, Vega-Lite: A Grammar of Interactive Graphics, 2017](#)
- [Documentation](#) et [galerie](#) Vega-Lite
- [Documentation](#) Altair
- [Documentation](#) Plotnine
- [Dépôt GitHub](#) Plotnine
- [Cours complet en notebooks](#) (Altair/Vega-Lite), par l'équipe à l'origine de Vega-Lite

Grammaires : cohérence et réutilisabilité

Une fois qu'un graphique est défini comme un mapping

(données $\rightarrow$ encodage)

changer de jeu de données ou ajouter une variable de facettage ne demande pas de réécrire le graphique !

On modifie un seul composant indépendant des autres.

Avec une approche impérative classique (tracer manuellement chaque élément), chaque variante redemande du code quasi entièrement nouveau.

Grammaire : sans JavaScript (Altair/Vega-Lite)

Certaines interaction complexes (filtrer ou créer un graphe juste en sélectionnant un ensemble de point dans un nuage) se code avec Altair en une dizaine de lignes déclaratives.

La même interaction écrite « à la main » en D3.js demande de gérer soi-même les événements souris, le recalcul des échelles et la synchronisation entre les deux vues.

C'est le compilateur Vega-lite qui s'occupe de tout pour nous.

Grammaires : conclusion

Les avantages de cette approche ne sont pas seulement conceptuels, il sont concrets :

- possibilité de « raisonner » sur un graphique avant de l'écrire,
- et de le faire évoluer sans tout reconstruire.

Pour aller plus loin :

- [Galerie Vega-Lite](#) (pour voir l'étendue de ce qu'une grammaire compacte permet d'exprimer)
- [Observable Plot](#) (même famille conceptuelle, côté JavaScript, par l'auteur de D3.js)

Grammaire : le notebook

[Vous y trouverez :](#)

- Une intro théorique qui reprend les éléments ci-dessus
- Une section sur Plotnine qui montre la logique et la facilité à créer certains graphes
- Une section sur Altair, pour comparer, qui affiche les même graphes avec les mêmes données, + un aperçu de ses capacités dans le domaine de l'interactivité
- Un comparatif des avantages / inconvénient de ces deux bibliothèques

Panorama des technos de dataviz

Vous trouverez sur le dépôt un fichier .md qui passe en revue les technos, modules, solutions, etc. pour faire de la data-viz.

Pour certaines j'ai ajouté des notebooks, guides, etc. issus de cours que j'ai donné sur ces solutions (cf. liste sur le readme).

En fin de documents sont centralisés des liens vers ces solutions (présentation, dépôts, doc, galerie... en fonction des ressources disponibles). À enrichir !

Ressources : histoire et bases de la dataviz

- [Friendly, A Brief History of Data Visualization](#)
- [data-to-viz.com](#)
- [Visual Vocabulary](#) (Financial Times)
- [Storytelling with Data](#)

Ressources : perception et cognition

- [Cleveland & McGill \(1984\)](#)
- [Miller \(1956\), *The Magical Number Seven*](#)
- [Gestalt psychology](#)
- [Amar, Eagan & Stasko \(2005\)](#)
- [Munzner, *Visualization Analysis and Design*](#)
- [Few, *Information Dashboard Design*](#)

Ressources : grammaires de visualisation

- [Wickham \(2010\), *A Layered Grammar of Graphics*](#)
- [Satyanarayan et al. \(2017\), Vega-Lite](#)
- [ggplot2](#)
- [bbplot](#) (BBC)
- Vega-Lite ([doc](#) / [exemples](#))
- [Altair](#)
- [Plotnine](#) ([dépôt](#))
- [UW Data Visualization Curriculum](#)
- [Observable Plot](#) (analyse exploratoire avec JS)

Ressources : outils de dataviz/dashboarding

Outre les cours présents sur le dépôt :

- [Metabase](#) ([doc](#) / [tutoriels](#))
- [Apache Superset](#) ([galerie](#) / [doc](#))
- [Dash](#) (Plotly) ([doc](#) / [galerie](#))

Ressources : outils de dataviz/dashboarding

- [Tableau](#) ([tutoriels](#) / [Tableau Public](#) - gratuit)
- [Power BI](#) ([tutoriels](#))
- [D3.js](#) ([galerie](#))
- [React](#) / [Vue](#)
- [Three.js](#)

Ressources : outils geodata

- [GeoPandas](#)
- [Shapely](#)
- [GDAL](#)
- [Cartopy](#)
- [Folium](#) / [Leaflet](#)
- [MapLibre](#) (GL JS)
- [kepler.gl](#) ([doc](#))